TD 1 - Comparing models on the same questions

Final · 2026-10-14-students-1, 14 October 2026

Final version for TD 1 and its preparation. This resource is stable until the session.

> Final. Pack 2026-10-14-students-1. Set your machine up with Before TD 1.

Module: Trustworthy generative AI. Session 1 (21 October), 14:00 to 18:30.

An assistant turns a revenue question into an SQL query, then into an amount. This session measures five free models on twenty common questions, against a reference that your sub-group computes itself. By 18:30, you know which model fails on which questions, for what cause, and where each model sits in a model portfolio: capability, price, access route.

What you can do by the end of the session

- Send an identical prompt to several models, so that a gap between them is attributable to the model and not to the wording.
- Run a query produced by a model in a read-only sandbox and give it a verdict by comparing its value with your reference.
- Read a trace table: for each failure, tell apart a broken convention, invalid SQL, a truncated output and a refused access.
- Place each model of the kit in the portfolio: public index, price per million tokens, access route.

Material provided

The `kit-td1` kit, one copy per member, files kept together:

- the database: `data/boutique.sqlite` (5 products, 59 customers, 221 orders over 397 lines), its three CSV files and `data/manifest.json` with their SHA-256 hashes;
- the twenty questions: questions-td1-en, and the `reference-td1.csv` template that you fill in;
- the protocol: consigne-etudiante-en (schema, conventions, accounts, call rules, authorised models, hand-in);
- the orchestrator skeleton `skeleton-multi-llm-en.py` and its sandbox `sandbox_sqlite.py`: common prompt, clients, read-only execution, verdict and trace table are provided; the vote is yours to write in TD 2;
- the portfolio: portefeuille-modeles-en and its CSV, sixty model and access pairs with public index, price and access route;
- access to the models: your personal Groq Free and OpenRouter accounts. Five required models (three Groq, two OpenRouter), exact identifiers and parameters in `modeles-autorises.json`; environment set up before the session with guide-environnement-etudiant-en and checked by `check_access.py`.

The kit and its documents are in English, the language of the module.

The two result templates and their fictional presentation example are included in the kit: model-question-comparison-en.csv and five-model-summary-en.csv.

Schedule

Times are working landmarks for the session. Take breaks as needed while coordinating with your sub-group.

Time	Step	Expected output
14:00 to 14:15	Check saved preparation evidence, review the workflow, assign roles	account allowances and allocation recorded
14:15 to 15:30	**Step 1**: twenty reference queries, shared among members	independently checked `reference-td1.csv`
15:30 to 16:05	**Step 2**: five models, allocation and troubleshooting included	raw CSV and JSONL traces, all attempts
16:05 to 16:30	**Step 3**: read the traces	final results and explanations
16:30 to 16:45	Suggested break window	
16:45 to 17:30	**Step 3**, continued: complete the two summaries	comparison CSV and five-model summary
17:30 to 18:00	**Step 4**: plenary of four sub-groups	comparison of the evidence
18:00 to 18:30	Mark the session and prepare TD 2	R1, R2 and R3 on the assessment sheet

---

Opening check (14:00 to 14:15)

Complete installation and preparation **by 18 October inclusive**. Bring the saved `check_access.py` output: local integrity checks and all five required accesses OK. Read the TD 1 assignment before arriving. Use the first few minutes to review the files and the sequence: references, calls, traces, summaries.

Check the saved evidence and remaining account allowances. Run `check_access.py --no-network` for local integrity if needed; a new network check consumes three Groq and two OpenRouter calls and is counted separately. Address any changed access with the instructor and record a fallback if necessary.

Share the twenty references among all eight members. Assign the five model batches to A-E as in the protocol; C and E use two different members' OpenRouter accounts. F-H cross-check, consolidate traces and prepare the summaries. Roles rotate in TD 2 and 3.

Step 1 - The reference (14:15 to 15:30)

The reference is the exact amount for each question, computed on the database by executing SQL and independently checking the result. Keep it independent of the answers being evaluated. Coding assistance is proposed with disclosure and verification, pending school approval; an LLM amount never supplies the reference.

Share the twenty questions of questions-td1-en out among the members, two or three each. For each question, write a SQLite `SELECT` that returns one row and one column `revenue_cents`, run it on `data/boutique.sqlite` (`sqlite3` command line or Python script). Cross-check it with a separate computation on the CSV files or an independent query, then record the query and its value in `reference-td1.csv`. The conventions in section 1 of the protocol fix the meaning of each question: period closed on the left and open on the right, completed age to the month and day, revenue in cents excluding VAT, 0 when nothing matches.

Explain the period, age, product and amount conventions, then independently cross-check each assigned query with another member. Show the twenty values to the instructor: as the database and the questions are common, they name the questions whose value differs from the answer key, without giving the value. Resolve any difference before running the affected questions. Correct results on the first attempt give access to R1 level 2 when the convention and independent check are explained.

Deliverable of the step: `reference-td1.csv`, twenty lines, one query and one value each, equal to the answer key.

Step 2 - The five models (15:30 to 16:05)

The base batch is **100 cells: 60 Groq and 40 OpenRouter initial calls**, shared as follows: A, GPT-OSS 120B on Groq; B, GPT-OSS 20B on Groq; C, Cohere North Mini Code on OpenRouter; D, Qwen on Groq; E, Dots on OpenRouter. Each handles 20 cells. Members C and E use two distinct personal accounts. The 14 October selection contains these five models.

Each operator checks their remaining allowance and aims to reserve five extra calls for any further controls and retries, where that allowance permits. Record initial calls, access checks and retries separately. If capacity is insufficient, split questions among members with remaining capacity or ask for identified fallback traces. A new key does not add capacity. The JSON budget and protocol carry this same rule.

Use the PowerShell environment-variable commands in the environment guide on Windows. The following command runs only A's model and two questions whose references have already been calculated and validated:

```
python3 skeleton-multi-llm-en.py --run --models groq:openai/gpt-oss-120b --cases
C01,C03 --state state-A.jsonl --out traces-A.csv
```

Adapt `--models` to the assigned exact provider/model pair; extend `--cases` to finish its questions. Pass the turn between operators: one request at a time per sub-group, three seconds between requests, temperature 0, capped output, 45-second timeout and at most one retry per cell across resumptions. Keep the state file to preserve that limit. The subgroup budget counts 100 first attempts, not 100 attempts including retries.

For each question, the skeleton sends the same prompt, executes returned SQL in the read-only sandbox and compares with your checked reference. Explain the common prompt fingerprint per question. A changed prompt or reference starts a separate experiment: new `--state` and `--out`, with the old files retained.

Deliverable: merge the raw CSV rows under one header into `traces-td1.csv`, retaining all attempts; also retain every source CSV and JSONL. Columns include provider, model, question, prompt fingerprint, SQL, value, reference, verdict, HTTP status, error, latency, tokens, native cost, attempt, date and nature. Missing measurements stay empty. Real, replayed and simulated rows remain distinguishable.

Step 3 - Read the traces and complete the summaries (16:05 to 16:30 and 16:45 to 17:30)

Fill model-question-comparison-en.csv, in UTF-8 CSV format, with one final row per model/question. Its columns are `case_id`, `model_id`, `trace_file`, `cell_id`, `nature`, `status`, `value_cents`, `reference_cents`, `explanation`. Take the final verdict from the last attempt; keep all attempts in the raw traces. An unfinished cell is `not_executed` with its reason. Explain an error in `explanation` or link a note there, using the SQL and error fields as evidence. A failed access differs from a wrong answer.

Display a models x questions pivot of this CSV at the plenary. **Submit the long CSV**: the matrix is a view, not a second manual entry. There is no third mandatory error table. The fictional example shows the format with invented identifiers and values.

Fill five-model-summary-en.csv, one row for each of the five required models: public index with source/date, documented input and output prices for a paid route of the exact same model, source/date of prices, access used, results by period/age/product and combined families, trace files and limits. Use the full portfolio as a reference; do not rewrite it. If no matching paid route is documented, enter N/A with the reason. The summary supports a routing hypothesis from this batch, not a general model ranking.

Deliverable: the two completed CSVs, with causes in the comparison rows or linked notes. Be ready to explain an instructor-selected trace, its fingerprint and a missing measurement or retry if present. A retry need not occur. If the batch has no failed access, an identified instructor-supplied historical or simulated example gives the same opportunity to demonstrate R3.

Step 4 - Plenary (17:30 to 18:00)

Each sub-group presents its table and its sheet. Same database, same questions, same models: the gaps between sub-groups come from the references, the call parameters, the accesses and the variability of the models. Compare the traces behind each gap. Note the questions that most models miss and the convention involved: they come back in TD 2.

The instructor then marks the session at each table: R1, R2 and R3, each 0, 1 or 2, on the sheet laid on your table since 14:00. Before leaving, each sub-group designates who prepares TD 2: reading `atelier-vote.py` and the expected routing rule.

What is expected (the evaluated artefacts)

The code is not graded for itself; the outputs are. Your sub-group hands in exactly these artefacts:

1. `reference-td1.csv`: the twenty reference queries and their values, equal to the answer key.
2. `traces-td1.csv`: one line per call, missing measurements left empty, the same prompt fingerprint on every line of a question.
3. `model-question-comparison-en.csv`: final results, explanations and raw-trace references; a pivot view is displayed.
4. `five-model-summary-en.csv`: the five models, documented index/prices/access and results by family.
5. The prompt, as used: system message and user template of the skeleton, with any changes made, identical for all models.

Hand-in and session report

At the end of TD 1, add `reports/td1.md` to your sub-group's public Git repository, shared from TD 1 to TD 3. Describe the work attempted and completed, successes and failures with links to the results and traces, observed causes and hypotheses, limitations, each member's contribution and next steps. Identify replayed or synthetic examples explicitly.

Commit the report with the code, prompts and evidence. Update your entry in the same Moodle hand-in area with the full repository URL, the session's full commit hash and a link to the report at that commit. Retain earlier TD entries and reports. Check that the repository opens without signing in. Follow the hand-in dates announced in Moodle.

What is valued

- **A reference equal to the answer key**, obtained with the conventions of the protocol.
- **Traces that point to real calls**, one line per call, distinct natures: a simulated or replayed line never enters a real count.
- **One cause per failure**, read from the trace, not an impression.
- **A portfolio sheet that prepares the decision**: which model is enough for which question family, at what price.